

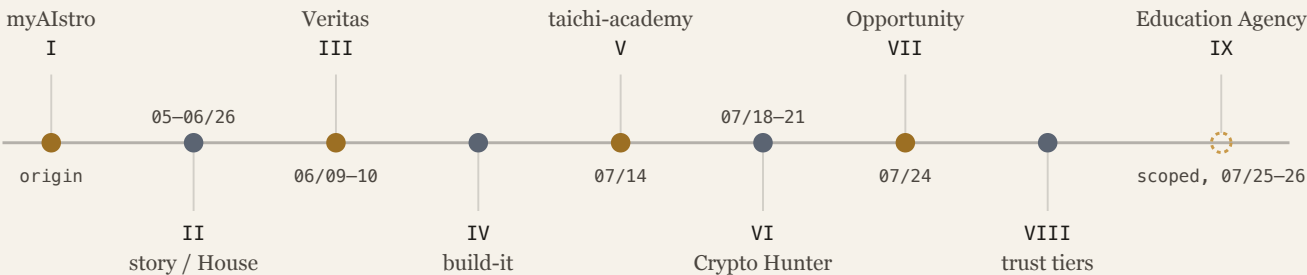
The deterministic scaffold: a case study in compounding architecture

How one invariant — the model proposes, a deterministic gate decides — generalized across nine checkpoints without ever being rewritten.

Author — 0ne29 Compiled by — Claude (Sonnet 5) Period covered — 2026-05-31 to 2026-07-25

ABSTRACT

This paper traces one architectural idea across nine checkpoints of real, shipped work, plus one still-scoped extension: the claim that reliability in an LLM system should live in a deterministic verification boundary, never in the model's judgment. The evidence for this claim is not that it sounds coherent — it is that the same invariant was re-implemented independently, under increasing stakes and across domains that share no code: a personal learning system, a story generator, a real-time heist simulation, a hosted teach-yourself-to-code reader, a 30-project physics curriculum, an autonomous-org framework, a crypto-opportunity hunter, a meta-org of hunters, and, as of this session, a still-unbuilt pair of "Manager" engines whose required shape was predicted a month in advance and confirmed by an unrelated line of reasoning. What follows is a synthesis of that record, compiled from persisted session memory rather than raw transcript, with dates, test counts, and live-run costs cited wherever they were recorded — and with the places the pattern does *not* uniformly hold named plainly rather than smoothed over.



00 A methodological note

This account is reconstructed from dated memory checkpoints written at the end of past sessions, not from a complete transcript archive. Where a figure is cited — a test count, a dollar cost, a date — it is because it was recorded at the time, not because it was reconstructed after the fact. Some checkpoints are 45–80 days old at time of writing; project state may have moved since. The paper's job is to trace the shape of the thesis's evolution, not to certify the current state of any one repository.

01 The claim

The Deterministic Scaffold: an LLM proposes; it never decides. A separate, non-LLM gate — code that runs a test, checks a domain registry, diffs a claim against a ground-truth document, enforces a hard numeric floor — is the only path from "proposed" to "trusted." The corollary that recurs at every checkpoint below, phrased slightly differently each time: *the grader can't be the thing it grades*.



02 Checkpoint I — the origin

myAIstro · earliest recorded instance in this history

The earliest instance on record is not a shipped "engine" at all — it is the user's own lesson-ingestion system, built to stop re-summarizing the same material on every query. Pasted lessons are summarized, then must clear a validation hard-gate before persisting to the source of truth; that gate has caught real failures repeatedly — truncated JSON, degenerate summaries, a fallback path leaking raw output. The design detail that matters most for everything that follows: **three separate local models by role** — one summarizes, a second (chosen specifically for a 128K context window) handles quiz generation and advisory queries, and a third judges, deliberately kept separate from both generators so the grader is

never grading its own work. This is the grader-can't-grade-itself rule, discovered before it had a name, enforced by model separation rather than a code gate — a variant worth naming on its own.

myAlstro's own architecture doc names this discipline directly — a section literally titled "The Deterministic Scaffold," with four numbered stages (Validation, Judge, Orchestrator, Memory Writer) — so the rule was conscious within this one project from early on. What wasn't yet visible is that the project kept re-deriving the same rule for every feature added to it since, independently of the other checkpoints in this paper. Its Classroom subsystem, a later addition, contains at least three more instances on its own:

Mastery-goal extraction — zero LLM involvement, by design:

```
Pure Python, no LLM. Single pattern, conservative on purpose.
[...]
- No model-assisted extraction. The methodology says "don't let
  the model guess." Pattern-match or return [].
```

backend/core/mastery_extractor.py, module docstring

Grading — a floor against one-question flukes, as a named constant, not a judgment call:

```
# Need at least this many CHECKs in a session for that session to
# count toward mastery. A single-CHECK lesson where you got lucky
# isn't mastery; a lesson with 4 CHECKs all first-try-correct is.
MASTERY_MIN_CHECKS = 2
```

backend/core/grading.py

Typing-practice beats — the LLM is excluded from the step, not merely checked:

```
# Deterministic post-processing: inject TYPING_PRACTICE beats from
# the entry's verbatim code_blocks (extracted during ingestion).
# The LLM never sees the typing-practice content — we don't trust
# any model to reproduce a code snippet character-perfectly, and
# the SOT already holds the canonical version.
```

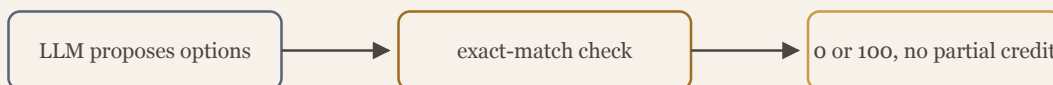
backend/agents/teacher_aide_agent.py

Raise-hand Q&A — a hard refusal, not a soft suggestion, when there's nothing to ground in:

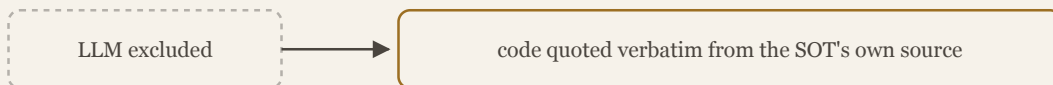
```
if not source_text.strip():
    # No source to ground against – refuse rather than let the
    # Teacher hallucinate. This is the same posture validation
    # takes at the SOT-write boundary.
    raise HTTPException(
        status_code=409,
        detail="Cannot resolve grounding source for this plan – refusing to answer ungrounded"
    )
```

backend/api/classroom_controller.py, session_raise_hand_endpoint

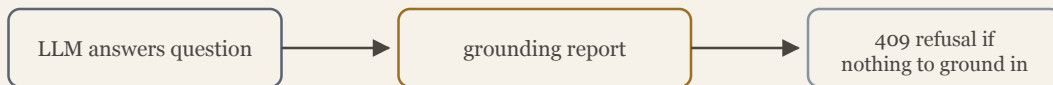
CHECK



TYPING_PRACTICE



RAISE-HAND



Three different shapes of the same discipline, in one subsystem: check-after (CHECK), exclude-instead-of-check (TYPING_PRACTICE), and check-with-a-hard-refusal-fallback (RAISE-HAND).

LOAD-BEARING DISTINCTION

None of these is "the" gate pattern — each is the pattern shaped by what the specific risk actually requires. The invariant held across this paper is fixed: an LLM's output is never self-certifying. The implementation is not fixed, and should not be — it varies with what kind of wrongness is actually possible. Read as one universal verification mechanism, this paper would be a much smaller and much less interesting claim than the one it is actually making.

A fourth instance is planned, not shipped, and the project is explicit about the difference: its own architecture doc lists "cross-lesson synthesis" — letting the Teacher Aide draw from

multiple related lessons instead of one — under "preparing for (not yet implemented)," beside an honestly-documented list of current sharp edges (MC distractor quality tracks model quality; severe JSON corruption can still under-produce a plan). The project holds itself to the same standard this paper does: state what's real, and what's still open, without blurring the two together.

03 Checkpoint II — the rule survives domain transfer

my-AI-story (shipped 2026-05-31, 88 tests) & The House Always Wins (begun 2026-06-08)

Two near-simultaneous instances, in domains that share nothing. My-AI-story: no episode is persisted or spoken until a pure-Python verifier confirms it matches the series bible; continuity, structure, and speaker checks are blocking, sound-cue verification deliberately is not — "sound is enhancement, continuity is the premise." The House Always Wins pushed the rule into a real-time simulation: a per-frame pipeline (assess → propose → verify → commit → log) where three named, pure verifier functions — `verify_blackout`, `verify_surveillance`, `verify_masking` — are the only path from a crew's proposed action to a committed one. Its own phrasing of the invariant: *"the model proposes; Python disposes."*

This checkpoint also produced a variant of the gate concept worth naming: a deliberately engineered floor of irreducible uncertainty. The heist's outcome is designed to never converge on certainty — trained crews were tuned to win roughly ninety percent of the time, never one hundred — because a gate that lets skill hit certainty stops being suspenseful and starts being scripted. And a smaller, easy-to-miss fact about how this work actually persists: the core simulation engine was originally built with Claude in a session from **December 2025**, months before this project existed by name — the artifact outlived that conversation entirely and was later adopted wholesale as this project's core. The thesis surviving past the memory of the session that produced it is itself a small piece of evidence for the thesis.

04 Checkpoint III — naming the substrate

Veritas · P0-P4 · 2026-06-09 to 2026-06-10 · 13 → 31 tests

Veritas extracted the pattern from checkpoint I and named it as a reusable primitive: a provider-agnostic model boundary (`propose(role, prompt) -> text`), a Gate abstraction

whose own class docstring states the whole thesis in one line —

```
"""The Gate — the decision engine.
```

```
A gate is a function `Artifact -> GateResult`. It is the only thing in Veritas
allowed to say yes or no. Deterministic wherever a real check exists; explicitly
SOFT where it doesn't. The LLM proposes; the gate decides.
```

```
"""
```

```
engine/gate.py, module docstring
```

P2 sharpened this further: acceptance requires at least one hard gate to pass and every hard gate to agree — *"cannot be accepted on judgment alone."* The clearest live proof of the thesis working as designed, not merely as described, came from an ordinary run: llama3.1:8b shipped a correct `factorial()` with a `ValueError` on negative input; the QA agent — itself an LLM, therefore honestly a soft gate — wrongly flagged it. The system recorded the QA dissent as a soft finding and did not block the hard-verified code. Nothing was hidden or discarded; the disagreement is on the record, and it didn't matter, because the hard gates had already decided.

P3 added a second, independent claim: that the organization could learn from its own failures without any weight update, purely through a deterministic recall mechanism over stored memory. `test_org_avoids_repeating_its_own_failure` is a real regression test, not a metaphor — a proposer fails uninformed, the failure is remembered, and the next attempt, warned by recall, succeeds. Commit hashes and a live test run: Appendix A.2.

05 Checkpoint IV — off the laptop

build-it · live at build-it-mauve.vercel.app

Every checkpoint before this one runs locally. Build-it is the first public, hosted instance — a calm "Lego manual for code" reader for absolute beginners, built by the same adult-beginner who is its own first audience. It is the cleanest demonstration yet that the gate does not need the LLM to exist at all: the deterministic reader, the step-by-step verifier, and even a fully deterministic setup interview (run a command, paste the output, it reads it) all work with **zero API key**. The one place an LLM appears — a page-aware helper that answers "check my code" or "something's not right" — only ever *voices* a verdict a deterministic contains-

check already reached, and degrades gracefully to the step's own recovery notes if no key is present. The thesis doesn't just generalize across domains; it generalizes across how much infrastructure is available to run it.

06 Checkpoint V — scale

taichi-academy · all 30 projects shipped 2026-07-14

A thirty-project GPU-simulation curriculum, each project built reference-first: a verified implementation and its tests exist before a single lesson fragment is written, and a deterministic checker (`tools/check_step.py`) — not the model — renders every verdict while a learner hand-types code. Thirty independent applications of the same discipline is a different kind of evidence than one deep case study: it is a claim about the pattern holding under repetition without drifting, and the record backs that up with specifics, not just a count — a raw five-point Laplacian that was unstable at a given timestep, a divergence/ gradient discretization mismatch that silently halved a fluid solver's effectiveness, an energy-balance climate model's bistable snowball state that had to be tuned around. Each is a real bug the gate structure caught and turned into a documented teaching beat, not a hypothetical.

07 Checkpoint VI — proof under real stakes

Crypto Hunter AI · Phase 0-4a · 2026-07-18 to 2026-07-21 · 15 → 47 tests

Crypto Hunter AI is the precise test of whether the thesis was a real invariant or an artifact of Veritas's own codebase. It is **not** built on Veritas's code — no shared `Gate` or `Artifact` class. It is an independent, parallel implementation of the same rule, under real money and real adversaries, later bridged into Veritas's registry read-only, never re-judged. That separation is the whole point: if the invariant only held inside one codebase, it would be a design pattern. Holding across a second, unrelated implementation is evidence it is something closer to a real constraint.

The gate laws in this checkpoint were not designed up front — they were found live, by real failures, and locked as regression tests the same day (named test functions and a live run: Appendix A.3):

- **Evidence must be monotonic.** A verifier-added source that fails validation is excluded from the count, never used to demote a candidate — except scam-listed domains, the deliberate exception, which hard-fail anywhere.
- **Negative evidence fails closed.** An RDAP outage on a scam-report domain must not disarm the skeptic; an unresolvable report domain is treated as a rejection, not a shrug.
- **Re-filing under a name variant cannot evade a scam report.** Reports attach to the normalized opportunity, and name-variant siblings inherit them at gate time.

Evidence is monotonic: a source that cannot be validated is EXCLUDED (never counted as a demotion) — same monotonicity standard applies to scam reports; a scam-listed domain anywhere is a hard fail.

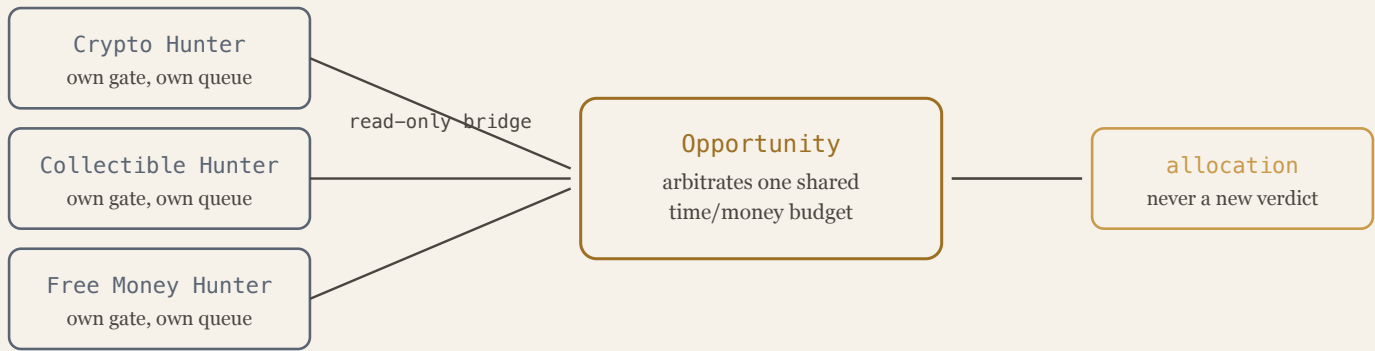
engine/gate.py, paraphrased inline from the check's own comments

The cost of running an autonomous multi-agent org against live data is on the record, not estimated after the fact: \$1.30 for the first full day-cycle, \$2.23 cumulative through Phase 1, later running at roughly \$0.12 per day-cycle once a build-mode provider swap was needed. The organization now runs on an unattended daily cron.

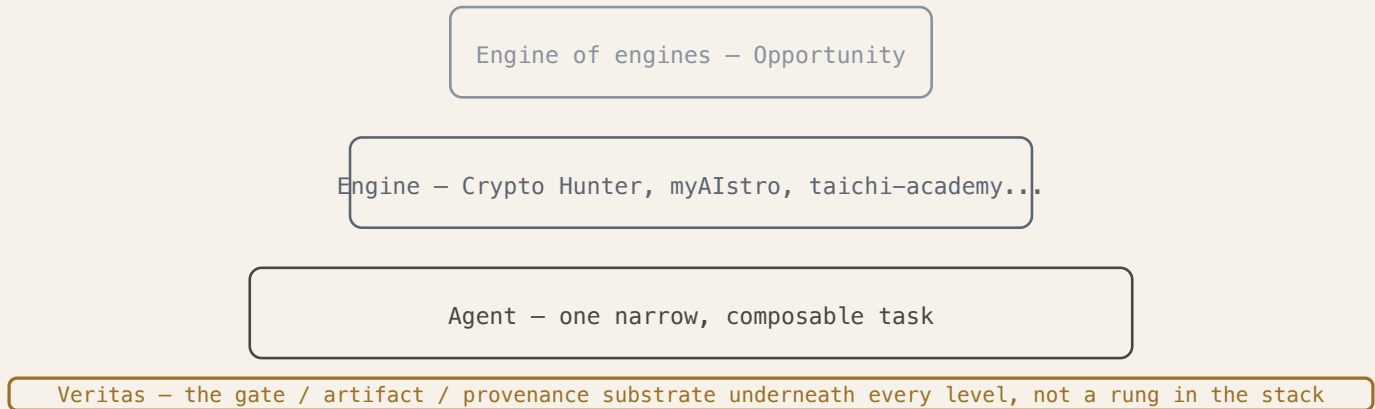
08 Checkpoint VII — recursion, one level up

Opportunity [Agency AI] · shipped 2026-07-24 · live-merged real data

Opportunity is not a new invariant — it is the same invariant applied to a new level of composition: *"an agent organization of agent organizations."* Crypto Hunter's reusable 80% was extracted into a shared substrate, `hunter-engine`, and two new independent engines — Collectible Hunter AI and Free Money Hunter AI — were built on it and live-validated the same week for roughly six and eleven cents respectively. Opportunity itself reads each engine's already-gated queue through a read-only bridge, never re-deciding a verdict, and arbitrates one shared time-and-money budget across all of them. Its first live run was not a simulation: eighteen real Crypto Hunter opportunities and five real Free Money Hunter opportunities were merged into one genuine cross-domain allocation. Commit hash and a live test run: Appendix A.4.



This checkpoint also produced the naming discipline that keeps the thesis from sprawling into an unbounded list of "projects": something only earns a new name if it is a new *level* or *kind* of composition. Applying the same gate/trust discipline one more time is not a new project — it is Veritas, reused.



09 Checkpoint VIII — the substrate refines itself

this session · 2026-07-25 · scoped, not built

Checkpoint IV's own memory left an explicit open question, deliberately not committed to: whether the Hunter archetype (a stateless discover-and-report loop) generalizes into a second archetype — a **Manager** engine that operates and grows an owned asset over time, needing "an action-executor with its own approval tier... and a metrics-feedback loop closing back into the decision engine." That note was written weeks before this session, with no date attached to when it would be revisited.

Today's brainstorm arrived at that same shape from a completely unrelated starting point — not "what does the open thread predict," but "what does managing a YouTube channel actually need." A YouTube Channel Manager built around the studio's own already-produced

content (*Not It Protocol*, *The Journal of Informal Human Protocols*) and a parallel Commerce Store case both surfaced the predicted action-executor-plus-approval-tier and the predicted metrics-feedback loop, independently. Two different lines of reasoning landing on the same structure is the same category of evidence as checkpoint III's independent re-implementation — convergence, not persuasion.

The session also produced a genuine refinement rather than a repetition: the gate-hard versus debate-argued *binary* from checkpoint IV generalizes into a four-tier trust spectrum —

0 · self-verifying	1 · consistency	2 · fact-bank	3 · floor + human
low stakes, no claim about the world		irreversible or outward-facing	
TIER	NAME	WHAT IT GUARDS AGAINST	EXAMPLE
0	self-verifying	nothing — no claim about the world is made	a diff, a state-machine transition
1	consistency check	a cheaply reversible style mismatch	title/thumbnail formatting
2	fact-bank grounding	a claim that could cause real harm if false	resume, product specs, order status
3	hard floor + human	an irreversible or outward-facing action	sending, spending, publishing

Its explanatory value: it is why Hunter engines lean on tiers two and three almost everywhere (nearly every component makes an externally-sourced claim), while Manager engines are mostly tier zero and one, with trust machinery concentrated at just the action layer. Per the naming discipline from checkpoint IV, this is not a sixth checkpoint sitting beside Opportunity — it is a refinement to Veritas's own substrate logic.

10 The wider catalog — where the pattern holds, and where it honestly doesn't

The nine checkpoints above are not the whole studio; they are the load-bearing ones. Several other shipped tools carry the identical shape — AccessGuard's deterministic axe-core scan with an LLM only voicing the branded report, amigurumi-ai's crochet-geometry engine validating every spec before an LLM interview can touch it, my-AI-scene's per-boundary

verifier gates, and my-AI-beats' CLAP-gated audio sections. SprinklerPro3D's quality-gate is worth naming specifically because it is currently *failing* on all five test yards — an honest, unresolved result, not a cherry-picked success, which is itself the kind of evidence this paper is trying to hold itself to.

Others in the catalog — Particle Life, universe-forge — are real-time physics and generative art, verified stable over long runs, but they do not carry an LLM-proposal-versus-gate split at all; there may be no LLM in the loop to distrust. Counting them as instances of this specific thesis would be the same kind of overclaim this paper is trying to avoid elsewhere. And my-AI-loop, a related idea for a conversational composer, is explicitly banked and unstarted — included here only because a thesis's honest edge includes the things it hasn't gotten to yet.

11 What "provable" means in this record

Every checkpoint above is cited with something falsifiable: a locked regression test born from a real bug, a dollar figure from an actual live run, a dissenting verdict left on the record rather than smoothed over, or an independent re-derivation landing on a predicted structure. None of it rests on an assertion that the architecture "feels right." That distinction is the thesis's own selection pressure applied to this paper: a claim about the work should be grounded the same way a claim inside the work is required to be.

WHAT WOULD NOT COUNT

A design that sounds elegant, a pattern applied uniformly "for consistency," or a finding manufactured to make a review look thorough. The record above deliberately excludes anything that could not survive being asked "grounded in what, exactly?"

One instance in this record is a sharper evidence class than the rest, worth separating out. Checkpoint VIII's Manager-archetype convergence is *convergent derivation* — two lines of reasoning, run in the same conversation, landing on the same structure. That is real, but it leaves a small gap: a session already primed with the pattern could, in principle, steer both lines toward each other without either being aware of it. Checkpoint I's Classroom subsystems close that gap entirely, because they are *retroactive fit*: mastery-goal grounding, typing-practice exclusion, and raise-hand's hard-refuse-if-ungrounded gate were all built and

shipped inside myAIstro weeks before "Education Agency AI" or the Manager-engine archetype were formulated as concepts at all. They could not have been aimed at a target that did not yet exist. When something built for its own local reason turns out to already satisfy a requirement invented later, the only explanation left standing is that the invariant was real independent of anyone naming it — a stronger claim than convergence can make on its own.

12 Limitations and open threads

- The persistence-layer name is not uniform across the lineage — Veritas calls it `MemoryStore`, predating the Hunter-family's `DataHub` naming — and that is stated plainly rather than smoothed into false consistency across the whole scaffold.
- The Manager archetype (Career, YouTube, Commerce) is scoped at the architecture level only — no code exists yet for any of the three.
- Veritas's hosted wedge has every primitive except billing, which remains explicitly deferred pending a real payment integration.
- The hardest tier of Opportunity's own engine roadmap — Property/foreclosures — is deliberately deferred; MLS licensing and county-level data fragmentation make it the least gate-clean domain surveyed so far.
- The four-tier trust spectrum has been checked against domains that share a common ancestor in this conversation. It has not yet been stress-tested against a domain nobody in this thesis has thought about yet.
- SprinklerPro3D's quality-gate is, as of its last recorded state, failing on all five test yards — included above deliberately, since a paper arguing for grounded evidence cannot omit its own unresolved failures.

13 Closing note

Veritas was never the top of this stack — it is the substrate every level sits on, the thing that makes an engine's output a checkable fact instead of a claim, whether the thing being checked is one agent's proposal or five engines' aggregate verdict. What this session added is not a new floor to build on top. It is a sharper description of how much weight the floor needs to bear at any given point — which, per its own logic, is exactly the kind of refinement

the substrate should keep earning, and exactly the kind it should stop earning the moment it stops being demonstrated this way.

14 Appendix A — independent evidence

Exhibits only, no narration. Every commit hash and test count below was read directly from the repository or run live during this compilation, not carried over from a prior summary. Where the underlying record has a real gap, that gap is named rather than filled.

A.1 · MYAISTRO — CLASSROOM COMMITS

```
be40354 classroom: TYPING_PRACTICE beat — verbatim-code typing drill
2432548 highlights (h2 fix): split multi-line green highlights into per-line mastery goals
fde0cc8 highlights (h2 mvp): user-authored "this matters" markers feed Classroom mastery_c
3712add H0 verification + tightened mastery binding (path A)
e123914 mastery goals (h0): deterministic per-lesson "what to master" field
```

```
git -C ~/myAIstro log --oneline, read 2026-07-26
```

A.2 · VERITAS — P0-P4 COMMITS + LIVE TEST RUN

```
5049021 Phase 4 — the Hub (local-first): control plane over the real engine
65dedae Phase 3 — the org learns: failure retrieval informs new work
7577fed Phase 2 — the cast: Security, QA, Validation, and hard/soft honesty
4feae34 Phase 1 — the first honest run: goal -> spec -> code, gated
4c81041 Phase 0 — the spine: Artifact, Gate, Memory, Run
```

```
def test_org_avoids_repeating_its_own_failure(tmp_path):    # tests/test_learning.py:79
```

```
$ .venv/bin/pytest -q
```

```
411 passed, 8 skipped (needs a running Docker daemon), 1 warning in 43.07s
```

```
git log + live pytest run, ~/MoreSalamander/veritas, 2026-07-26
```

A.3 · CRYPTO HUNTER AI — GATE-LAW COMMITS, TEST NAMES, LIVE RUN

```
9bc3225 gate v1.1: monotonic evidence semantics + RDAP registrable-domain fix
d82b776 phase 1: scam detection + scoring rubric + ranking + outcome tracking
6bd996f phase 2: daily mission (profile-driven, deterministic) + org dashboard
0cfb9e5 provider abstraction: OpenAI build-mode + two gate laws captured live
```

42886c3 phase 3: the debate goes live + 4 new scout beats + Veritas org-ification
 b6f510c phase 4a: on-chain sense (keyless) + daily autonomous run

```
def test_evidence_is_monotonic(): # tests/test_gate.py:5
def test_scam_taint_is_the_monotonicity_exception(legit_spec): # tests/test_gate.py:
def test_rdap_uses_registrable_domain(): # tests/test_gate.py:
def test_scam_reports_follow_the_opportunity_across_name_variants(): # tests/test_scoring.
```

```
$ .venv/bin/pytest -q
47 passed in 0.11s
```

git log + live pytest run, ~/MoreSalamander/crypto-hunter, 2026-07-26

A.4 · OPPORTUNITY – META-LAYER COMMIT + LIVE RUN

2c67b92 The meta-layer, v1: read-only bridges + deterministic cross-engine allocation

```
def test_shared_budget_is_genuinely_shared(): # tests/test_mission.py:19
def test_picks_across_engines_by_score(): # tests/test_mission.py:5
def test_empty_engines_produce_an_honest_empty_allocation(): # tests/test_mission.py:47
```

```
$ .venv/bin/pytest -q
5 passed in 0.01s
```

git log + live pytest run, ~/MoreSalamander/opportunity-agency-ai, 2026-07-26

A.5 · BUILD-IT & TAICHI-ACADEMY – COMMITS

build-it:

d5f84c3 Add the setup interview – the front door for true beginners
 3f2efe4 Finish the page-aware helper (chat phase 2)
 99da953 Make the six fireworks visibly distinct + vary explosion heights

taichi-academy:

212ba4a 30-universe-sandbox: direct N-body gravity – galaxies, black holes, collisions
 a575620 29-earth-simulator: energy-balance climate with seasons, ice caps, a biosphere
 831e097 28-digital-brain: spiking-neuron sheet, travelling waves, Hebbian plasticity
 92bf198 26-destruction: breakable-bond city that fractures under explosions and quakes

git log, both repos, 2026-07-26

A.6 · THE HOUSE ALWAYS WINS – JS TEST SUITE (FILE LISTING, VERIFIED PRESENT)

```
game/test/core.test.mjs
game/test/detection.test.mjs
```

```
game/test/guards.test.mjs
game/test/mechanics.test.mjs
game/test/navigation.test.mjs
game/test/obstacles.test.mjs
game/test/staff.test.mjs
```

```
ls ~/MoreSalamander/the-house-always-wins/game/test/*.mjs
```

A.7 · DOCUMENTED EXAMPLE FAILURES (THE GATE CATCHING SOMETHING REAL)

- Veritas P2: the QA agent wrongly flagged a correct `factorial()` implementation; recorded as a soft dissent, did not block the hard-verified code.
- Crypto Hunter, live day one: a scam-listed domain and an unresolvable RDAP report both had to be handled as hard-fail and reject-not-shrug respectively — both are now the regression tests in A.3.
- taichi-academy project 01: a raw five-point Laplacian was unstable at the lesson's chosen timestep — fixed with a nine-point stencil, documented as a teaching beat, not silently patched.
- taichi-academy project 02: a collocated central-difference divergence/gradient pairing was inconsistent with the solved Laplacian, silvering the fluid projection to roughly 36% effective before the fix.

HONEST GAPS IN THIS EVIDENCE BASE

No screenshots are included — none were captured for this paper, and a described-but-unseen screenshot is not evidence, so none is fabricated in its place. No CI logs appear because none of these repos run one; verification is local pytest plus live runs, cited above as such. And myAIstro's Classroom subsystem — unlike Veritas, Crypto Hunter, and Opportunity — has no automated regression suite backing it; its correctness claim rests on a single manual end-to-end script (`scripts/verify_mastery_goals_h0.py`), which is weaker evidence than a pytest suite and is named as such rather than presented at the same strength as A.2–A.4.

Compiled from: `project_myastro`, `project_myastory`, `project_housealwayswins`, `project_buildit`, `project_taichi_academy`, `project_veritas`, `project_cryptohunter`, `project_opportunity_agency`,

project_scaffold_recursion, project_trust_tiers, project_datahub_hackathon,
project_moresalamander – session memory, MoreSalamander StudioLabs, 2026-07-25.